

CRM Website — Planning Documentation

1. Project overview

A web-based CRM (Customer Relationship Management) system with:

- User authentication (login, signup, roles)
- Core CRM features (contacts, companies, deals/pipeline, tasks, notes)
- API connections with third-party tools (email, calendar, messaging, payments, automation)
- Supabase as the backend (Postgres database, Auth, Storage, Edge Functions, Realtime)

Assumptions made for this plan (adjust as needed):

- Multi-tenant SaaS-style CRM (multiple organizations/companies use the same app, each with isolated data)
- Team-based access — an organization has multiple users with roles (Owner, Admin, Sales rep)
- Frontend: React-based framework (Next.js recommended) — but the backend plan is framework-agnostic
- **Multinational usage:** users/teams operate across multiple countries, so currency, language, timezone, and data-residency need to be handled from day one rather than retrofitted (see Section 2.4)

2.4 Multinational requirements (added on top of the base plan)

These touch the schema, the UI, and infra choices, so it's worth deciding early:

Concern	Approach
Language / i18n	Store all UI strings in translation files (e.g. <code>next-intl</code> or <code>react-i18next</code>). Add a <code>locale</code> column on <code>users</code> and <code>organizations</code> (default locale for the org, per-user override). Don't hardcode English strings anywhere in components.
Currency	Deals need a <code>currency</code> column (ISO 4217 code) alongside <code>value</code> , not just a global default. Store amounts as integers in the smallest unit (cents) to avoid float rounding errors. Show an org-level "reporting currency" plus per-deal currency, and convert for dashboards using a rates table (updated via a scheduled Edge Function calling an FX API).
Timezones	Store all timestamps in UTC in Postgres (default behavior). Store each user's timezone (<code>IANA</code> string, e.g. <code>Asia/Kolkata</code>) and convert for display only in the frontend. Task due dates/reminders should be timezone-aware, not naive.
Phone numbers & formats	Store phone numbers in E.164 format (<code>+91XXXXXXXXXX</code>), format for display per locale using a library like <code>libphonenumber-js</code> . Dates: store as ISO 8601, format per locale on render.
Data residency / compliance	If customers are in the EU, UK, India, etc., you may need data to stay in-region (GDPR, and increasingly local data laws). Supabase lets you pick a project region at creation — for strict residency requirements, this can mean separate Supabase projects per region rather than one global project, with the frontend routing users to the right one. Worth deciding early since it's hard to migrate later.
Legal/compliance frameworks	GDPR (EU), and depending on target markets: CCPA (California), POPIA (South Africa), PDPA (Singapore/others). This affects: consent tracking, right-to-delete flows, data export, and where OAuth tokens/PII are stored.
Address formats	Don't assume a fixed set of fields (state/zip aren't universal) — use a flexible address structure or a library like Google's <code>libaddressinput</code> patterns.
Support hours / SLAs (if this is sold as SaaS)	Not a technical concern per se, but worth flagging for the product plan — multinational customers may expect follow-the-sun support.

Practical impact on the schema (Section 4): add `currency` to `deals`, `locale` and `timezone` to `users`, and consider whether `organizations` needs a `region` column to route to the correct Supabase project if you go the data-residency route.

2. Core feature set

2.1 MVP (Phase 1)

- Auth: signup/login (email+password, Google OAuth), password reset, email verification
- Organizations & team members (invite users, assign roles)
- Contacts (people) and Companies (organizations)
- Deals/Pipeline (Kanban-style stages: Lead → Qualified → Proposal → Won/Lost)
- Tasks & reminders (linked to contacts/deals)
- Notes & activity timeline (calls, emails, meetings logged per contact)
- Basic dashboard (deal counts, pipeline value, recent activity)
- Search & filters

2.2 Phase 2 — Integrations

- Email integration (Gmail/Outlook sync — send/receive tied to contacts)
- Calendar integration (Google Calendar/Outlook Calendar — meetings tied to deals)
- Messaging integration (WhatsApp Business API, Slack notifications)
- Automation/webhooks (Zapier/Make.com compatible webhooks, custom triggers)
- Payment integration (Stripe/Razorpay — invoices, subscription billing if CRM itself is sold as SaaS)
- Import/export (CSV import of contacts, export reports)

2.3 Phase 3 — Advanced

- Reporting & analytics dashboard (conversion rates, sales forecasting)
 - Custom fields & custom pipelines per organization
 - Role-based permission granularity (field-level access)
 - Public API + API key management (for customers to integrate their own tools)
 - Audit logs
 - Email/SMS campaign tools
-

3. Authentication & authorization plan (Supabase Auth)

3.1 Auth methods

- Email + password (with email confirmation)
- Google OAuth (and optionally Microsoft, for Outlook users)
- Magic link (optional, passwordless)

3.2 Session handling

- Supabase Auth issues JWT access tokens + refresh tokens
- Store session in httpOnly cookies (via `@supabase/ssr` if using Next.js) rather than localStorage, to reduce XSS risk
- Middleware on protected routes to verify session server-side

3.3 Authorization model (multi-tenant)

- Every table with org-owned data has an `organization_id` column
- **Row Level Security (RLS)** policies on every table restrict reads/writes to rows where `organization_id` matches the user's org membership
- A `memberships` table maps `user_id ↔ organization_id ↔ role` (owner/admin/member)
- Example RLS policy pattern:

```
sql

create policy "org members can view contacts"
on contacts for select
using (
  organization_id in (
    select organization_id from memberships where user_id = auth.uid()
  )
);
```

- Role checks (e.g. only admins can delete) enforced via policy conditions checking the `role` column, or via a Postgres function `is_org_admin(org_id)`

3.4 API key management (for the CRM's own public API, Phase 3)

- Separate `api_keys` table (hashed keys, scoped to an organization)
- Validated inside Edge Functions before allowing external programmatic access

4. Database schema (Supabase / Postgres)

Core tables:

Table	Key columns	Purpose
<code>organizations</code>	id, name, created_at	Tenant/company using the CRM
<code>memberships</code>	user_id, organization_id, role	Links auth users to orgs with a role
<code>contacts</code>	id, organization_id, name, email, phone, company_id, owner_id	People
<code>companies</code>	id, organization_id, name, domain, industry	Organizations the contacts belong to
<code>deals</code>	id, organization_id, contact_id, company_id, pipeline_id, stage_id, value, status, owner_id	Sales opportunities
<code>pipelines</code>	id, organization_id, name	Custom pipelines
<code>pipeline_stages</code>	id, pipeline_id, name, order	Stages within a pipeline
<code>tasks</code>	id, organization_id, related_to (contact/deal), title, due_date, assigned_to, status	To-dos
<code>notes</code> / <code>activities</code>	id, organization_id, related_to, type (call/email/meeting/note), content, created_by	Timeline entries
<code>integrations</code>	id, organization_id, provider, access_token (encrypted), refresh_token, status	Connected third-party accounts
<code>webhooks</code>	id, organization_id, url, event_type, secret	Outbound webhook subscriptions
<code>api_keys</code>	id, organization_id, hashed_key, scopes, created_at	For public API access

Use `mermaid.js` ERD or Supabase's schema visualizer to diagram this once tables are finalized — happy to generate one if useful.

Sensitive data: OAuth tokens for integrations (Gmail, WhatsApp, Stripe) should be encrypted at rest — use Supabase Vault or an Edge Function that encrypts before insert.

5. API integration architecture

5.1 Pattern

- **Inbound (data into CRM):** Third-party services push data via webhooks → received by a Supabase Edge Function → validated (signature check) → written to Postgres.
- **Outbound (CRM to third-party):** Edge Functions hold the integration logic (e.g. "send email via Gmail API") and are called from the frontend or triggered by Postgres triggers/cron (Supabase's `pg_cron`).
- **OAuth connections:** Each integration (Gmail, Outlook, WhatsApp, Slack) uses OAuth2 — an Edge Function handles the callback, exchanges the code for tokens, and stores them (encrypted) in the `integrations` table.

5.2 Example integrations to plan for

Integration	Use case	Auth type
Gmail / Outlook	Sync emails to contact timeline, send from CRM	OAuth2
Google Calendar	Schedule meetings tied to deals	OAuth2
WhatsApp Business API	Message contacts	API key / Meta OAuth
Slack	Internal notifications (new deal, task due)	OAuth2 / Incoming webhook
Stripe	Billing (if CRM is sold as SaaS) or invoicing customers	API key
Zapier / Make	Let customers build their own automations	Webhooks + API key

5.3 Public API for customers (Phase 3)

- Expose REST endpoints via Supabase's auto-generated PostgREST API (already available) for read access, protected by RLS + API key
 - For custom logic beyond simple CRUD, wrap in Edge Functions and document with OpenAPI
-

6. Tech stack summary

Layer	Choice
Frontend	Next.js (React) + Tailwind CSS
Backend	Supabase (Postgres, Auth, Storage, Edge Functions, Realtime)
Hosting (frontend)	Vercel
Background jobs / scheduled tasks	Supabase <code>pg_cron</code> + Edge Functions
File storage	Supabase Storage (attachments, avatars)
Email sending	Resend / Postmark (transactional) + Gmail API (user-linked sync)
Payments (if applicable)	Stripe

7. Security checklist

- RLS enabled on **every** table with org-scoped data — never rely on frontend filtering alone
- Encrypt OAuth tokens and API keys at rest
- Validate all webhook payloads with signature verification
- Rate-limit public API endpoints (Edge Function middleware or Supabase's built-in rate limiting)
- Use httpOnly cookies for session tokens, not localStorage
- Principle of least privilege for roles (member vs admin vs owner)
- Audit log table for sensitive actions (delete contact, change role, export data)

8. Suggested build roadmap

1. **Foundation** — Supabase project setup, auth, organizations/memberships, RLS baseline
2. **Core CRM** — Contacts, companies, deals, pipeline UI, tasks, notes
3. **Dashboard** — basic reporting, activity feed
4. **Integrations round 1** — Email + calendar sync
5. **Integrations round 2** — Messaging (WhatsApp/Slack), webhooks, Zapier

6. **Public API + billing** — if CRM will be offered as a SaaS product to others

7. **Polish** — custom fields, advanced permissions, audit logs, analytics

9. Open questions to nail down before building

- Is this CRM for internal use only, or will it be sold as a multi-tenant SaaS to other businesses?
- Which specific third-party tools are must-haves for launch vs later?
- Do you need mobile apps, or is a responsive web app sufficient initially?